13 · Solution Step 3: The Projection Head

1. Goal of this step

Turn n LLM vectors of size 4,096 into **one** vector of size 1,024, with length 1, living in the teacher's embedding space.

2. What the paper does

Four parts, in order:

Part	What it does	Fixes which mismatch	Real sizes	Math file
a. Input projection	Multiply each token vector by a learned matrix: 4096 → 1024.	Wrong size (first pass)	$\mathbf{W}_{\text{in}}$: 4096×1024	`math/02`
b. Transformer encoder	Add learned position vectors, then 2 transformer layers (8 heads, pre-norm, FFN size 4096) let tokens exchange information.	Causal "early token" problem; wrong space	2 layers, $d_m = 1024$	`math/03`
c. Mean pooling	Average the real (non-padding) token vectors into one.	Wrong count	—	`math/04`

Part	What it does	Fixes which mismatch	Real sizes	Math file
d. Output projection + L2 normalize	One more learned matrix to the teacher's width, then divide by length.	Wrong size (final); makes dot product = cosine	$\mathbf{W}_{\text{out}}$: 1024×1024	`math/05`

After that, searching is just a dot product with each stored document vector (`math/06`).

Total size: about 25M parameters according to the paper (a rough count from the listed sizes gives closer to 30M; either way it is tiny, about 0.3–0.4% of the 8B LLM).

3. Why it is done this way

Design choice	Reason
Shrink to 1,024 *first*	The transformer layers are then cheap. Attention costs grow with width squared.
A small transformer, not just a linear map	Tokens need to share information before averaging. It also gives enough flexibility to "rotate" one space into another.
Only 2 layers, dropout 0	The heavy understanding is already in the LLM. The head is a translator, not a thinker.
Mean pooling	Simple and stable; every token contributes. Alternatives (last token only, attention pooling like NV-Embed) weren't tested in the paper.
L2 normalization at the end	The index compares with cosine similarity. Unit-length vectors make "dot product" and "cosine" the same thing, and stop scores from depending on how long the vector happens to be.

Design choice	Reason
Positional embeddings start at zero	Training begins with no position bias and learns one only if it helps. (The worked example's zero positions are exactly this starting point.)

4. Which problem it answers

- **Problem 1:** replaces an 8B embedder with a ~25M head.
- **Problem 2:** 2.0 ms instead of 43.5 ms.
- **Problem 3:** its input is the context-rich hidden states, not text.

5. What could go wrong

- **Too small to translate everything.** Rare terms and long coreference chains are hypothesized to be where it fails (file `18`).
- **Padding leaks.** If padding isn't masked out of the average, the vector gets pulled in a meaningless direction (`math/04` shows this with numbers).
- **Forgetting to normalize** makes scores depend on vector length and breaks comparison with the index (`math/05`, `math/06`).